Pimpri-Chinchwad Education Trust's
Pimpri-Chinchwad College of Engineering
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune University (SPPU) ISO 21001:2018 Certified by TUV

DEPARTMENT OF INFORMATION TECHNOLOGY

Assignment No. 8

Subject	DBMS Lab
Assignment No.	8
Topic	No SQL: Aggregation and Indexing.
Student Name	Madhav Khobare
Roll Number	124B1F023
Date	15/04/2026
Div & Batch	SY IT A1
Guided by: -	Ms. Alpana A. Borse

Academic Year: 2025-26 Semester: IV

Aim:

To design and develop MongoDB queries using aggregation and indexing.

Objectives:

- To understand aggregation in MongoDB.
- To perform operations like $group, $match, $sort, $avg, $sum.
- To understand indexing and improve query performance.
- To create and use indexes in collections.

Case Study: Airline Management System

MongoDB Queries (Aggregation & Indexing):

---

1. Aggregation using $group
Theory:

$group is used to group documents and perform aggregate calculations like count or sum.

```
db.flight.aggregate([
  {
    $group: {
      _id: "$source",
      total_flights: { $sum: 1 }
    }
  }
])
```

Output:

```
{ "_id": "Mumbai", "total_flights": 1 }
{ "_id": "Pune", "total_flights": 1 }
{ "_id": "Dubai", "total_flights": 1 }
```

---

2. Aggregation with $match
Theory:

$match filters documents before grouping.

```
db.flight.aggregate([
  { $match: { price: { $gt: 5000 } } }
])
```

3. Aggregation with $avg

```
db.flight.aggregate([
  {
    $group: {
      _id: null,
      avg_price: { $avg: "$price" }
    }
  }
])
```

---

4. Aggregation with $sort

```
db.flight.aggregate([
  { $sort: { price: -1 } }
])
```

---

5. Aggregation with $project

```
db.flight.aggregate([
  {
    $project: {
      flight_name: 1,
      price: 1,
      _id: 0
    }
  }
])
```

---

6. Aggregation Pipeline (Combined)

```
db.flight.aggregate([
  { $match: { price: { $gt: 4000 } } },
  {
    $group: {
      _id: "$source",
      avg_price: { $avg: "$price" }
    }
  }
])
```

---

7. Create Index
Theory:

Indexing improves search performance by creating a data structure for faster retrieval.
db.flight.createIndex({ flight_name: 1 })

---

8. Create Compound Index

db.flight.createIndex({ source: 1, destination: 1 })

---

9. View Indexes

db.flight.getIndexes()

---

10. Drop Index

db.flight.dropIndex({ flight_name: 1 })

Case Study: Civic Voice Interface System

MongoDB Queries (Aggregation & Indexing):

1. Aggregation using $group

```
db.complaint.aggregate([
  {
    $group: {
      _id: "$status",
      total: { $sum: 1 }
    }
  }
])
```

2. Aggregation with $match

```
db.complaint.aggregate([
  { $match: { status: "Pending" } }
])
```

3. Aggregation with $sum

```
db.complaint.aggregate([
  {
    $group: {
      _id: "$state",
      total_complaints: { $sum: 1 }
    }
  }
])
```

4. Aggregation with $sort

```
db.complaint.aggregate([
  { $sort: { complaint_id: -1 } }
])
```

5. Aggregation with $project

```
db.complaint.aggregate([
  {
    $project: {
      issue: 1,
      status: 1,
      _id: 0
    }
  }
])
```

---

6. Aggregation Pipeline

```
db.complaint.aggregate([
  { $match: { status: "Pending" } },
  {
    $group: {
      _id: "$state",
      total: { $sum: 1 }
    }
  }
])
```

---

7. Create Index

```
db.user.createIndex({ name: 1 })
```

---

8. Create Compound Index

```
db.user.createIndex({ state: 1, name: 1 })
```

---

9. View Indexes

```
db.user.getIndexes()
```

---

10. Drop Index

```
db.user.dropIndex({ name: 1 })
```

Conclusion:

In this assignment, MongoDB aggregation operations such as $group, $match, $sort, $avg, and $project were implemented to analyze data efficiently. Indexing techniques were also applied to improve query performance. This demonstrates how NoSQL databases handle large datasets effectively in real-world applications.